



WEEK-10 NOTES

▼ What is ASP.NET Web API

ASP.NET Web API is a framework used to build **HTTP-based RESTful services** on the **.NET Framework**.

It allows:

- Client apps (browser, mobile, Angular, React)
- To communicate with server
- Using **HTTP methods**
- And exchange data in **JSON / XML**

HTTP Methods Used

Method	Purpose
GET	Read data
POST	Create data
PUT	Update full resource
PATCH	Partial update
DELETE	Delete resource

What is a Controller?

A **controller**:

- Handles incoming HTTP requests
- Contains action methods
- Returns HTTP responses

```
public class ValuesController : ApiController
{
```

```
public IHttpActionResult Get()
{
    return Ok("Hello Web API");
}
```

What is IHttpActionResult?

It represents an **HTTP response**.

Common results:

- `Ok()` → 200
- `NotFound()` → 404
- `BadRequest()` → 400
- `Unauthorized()` → 401

FOLDER STRUCTURE

Solution: `CORSDemo`

This is your **Visual Studio solution** containing **1 project**.

Project: `CORSDemo`

This is an **ASP.NET Web Application**..

App_Start

This folder contains **application startup configuration**.

`WebApiConfig.cs`

Purpose: Configure Web API routing & CORS

```
config.MapHttpAttributeRoutes();
```

- Enables attribute routing
- Default route:

```
api/{controller}/{id}
```

RouteConfig.cs

Used for **MVC routing**, NOT Web API

```
routes.MapRoute(  
    name:"Default",  
    url:"{controller}/{action}/{id}"  
);
```

FilterConfig.cs

Used to register **global filters**

Examples:

- Exception filters
- Authorization filters

BundleConfig.cs

Used for:

- CSS bundling
 - JS minification
- (MVC feature, not Web API specific)

Controllers

This folder contains **controllers**.

ValuesController.cs

Web API Controller

```
public class ValuesController :ApiController
```

- Inherits from `ApiController`
- Handles API requests

- Returns data (JSON)

HomeController.cs

MVC Controller

```
public class HomeController : Controller
```

- Returns Views
- Used for UI (HTML pages)

Important:

Web API controllers → ApiController

MVC controllers → Controller

Models

Contains:

- Data models
- DTOs
- Request / response classes

Example:

```
public class Product
{
    public int Id {get;set; }
    public string Name {get;set; }
}
```

Views

MVC Views (HTML, Razor)

- Used by **HomeController**
- NOT used by Web API

Scripts

JavaScript files

- jQuery
- Bootstrap JS
- Frontend logic

Content

CSS files

- Bootstrap CSS
- Site styles

App_Data

Used for:

- Local DB
- XML files
- Temporary data

Global.asax

Application entry point

```
protected void Application_Start()
{
    GlobalConfiguration.Configure(WebApiConfig.Register);
}
```

This is where:

- Web API starts
- Routes are registered
- CORS config loads

Web.config

Application configuration file

Contains:

- Connection strings
- Authentication settings
- CORS headers (optional)
- Compilation settings

packages.config

List of NuGet packages

▼ What is CORS

CORS = Cross-Origin Resource Sharing

CORS is a **browser security mechanism** that controls **which frontend origins** are allowed to call your **Web API**.

What is an “Origin”?

An **origin** is made of **3 parts**:

Protocol + Domain + Port

Example:

https://localhost:4200

URL	Origin
https://example.com	https + example.com + 443
<u>http://example.com</u>	Different (protocol changed)
https://example.com:8080	Different (port changed)

How CORS Works (Technically)

Case 1: Simple Request (GET)

1. Browser sends request:

```
GET /api/users
Origin: http://localhost:4200
```

2. Server response:

```
Access-Control-Allow-Origin: http://localhost:4200
```

3. Browser checks:

- Origin allowed?
- Then response is delivered to JS

Case 2: Non-Simple Request (Preflight)

Requests that trigger **preflight**:

- POST with JSON
- PUT / DELETE
- Custom headers

Step 1: Browser sends OPTIONS (Preflight)

```
OPTIONS /api/users HTTP/1.1
Host: api.example.com
Origin: http://localhost:4200
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type,Authorization
```

Browser asks:

| "Can I send POST with these headers?"

Step 2: Server validates preflight request

- Checks allowed origins
- Checks allowed methods
- Checks allowed headers

Step 3: Server responds to OPTIONS

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:4200
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Max-Age: 600
```

This response **does NOT** contain actual data

Step 4: Browser evaluates preflight response

Browser checks:

- Origin allowed
- Method allowed
- Headers allowed

If any check fails → request is blocked

Step 5: Browser sends actual request

```
POST /api/users HTTP/1.1
Origin: http://localhost:4200
Content-Type: application/json
Authorization: Bearer token
```

Step 6: Server sends actual response

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: http://localhost:4200
Content-Type: application/json
```

Step 7: Browser delivers response to JavaScript

JS receives response successfully

Enable CORS in .NET Framework Web API (STEP BY STEP)

Step 1: Install CORS Package

Open **NuGet Package Manager** and install:

```
Microsoft.AspNet.WebApi.Cors
```

Step 2: Enable CORS in WebApiConfig.cs

 **App_Start/WebApiConfig.cs**

This is Global CORS settings

```
using System.Web.Http;
using System.Web.Http.Cors;

public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        // Enable CORS globally
        var cors = new EnableCorsAttribute(
            origins:"http://localhost:3000",
            headers:"*",
            methods:"*"
        );

        config.EnableCors(cors);

        // Web API routes
        config.MapHttpAttributeRoutes();

        config.Routes.MapHttpRoute(
            name:"DefaultApi",
            routeTemplate:"api/{controller}/{id}",
            defaults:new { id = RouteParameter.Optional }
        );
    }
}
```

```
    );  
  }  
}
```

This allows:

- Requests from `http://localhost:3000`
- All headers
- All HTTP methods

Step 3: Enable CORS Attribute on Controller

```
using System.Web.Http;  
using System.Web.Http.Cors;  
  
[EnableCors(  
    origins:"http://localhost:3000",  
    headers:"*",  
    methods:"*" )]  
public class TestController : ApiController  
{  
    [HttpGet]  
    public IHttpActionResult Get()  
    {  
        return Ok("CORS enabled successfully");  
    }  
}
```

Enable CORS Globally (Alternative)

Instead of controller-level:

```
config.EnableCors();
```

Then apply `[EnableCors]` where required.

Special Scenarios

Allow Multiple Origins

```
origins:"http://localhost:3000,http://localhost:4200"
```

Allow All Origins

```
[EnableCors(origins: "*", headers:"*", methods:"*")]
```

CORS with Authorization Header (JWT)

```
[EnableCors(  
  origins:"http://localhost:3000",  
  headers:"Content-Type,Authorization",  
  methods:"*" )]
```

▼ AUTHENTICATION & AUTHORIZATION

Authentication verifies the identity of a user.

Authentication Flow

```
Client → sends credentials  
Server → verifies credentials  
Server → creates identity (user context)
```

Authentication in Web APIs

Web APIs are **stateless**, so:

- Server does NOT remember users
- Each request must carry proof of identity

This is why **JWT tokens** are used.

Authorization decides what an authenticated user is allowed to do.

Authorization Happens AFTER Authentication

```
Request
↓
Authentication (Who?)
↓
Authorization (Allowed?)
↓
Controller Action
```

Authorization Types in .NET Framework

1. Role-based

```
[Authorize(Roles = "Admin")]
```

1. Policy-based (limited in Framework)

2. Custom Authorization Filters

Why JWT?

Traditional session-based auth doesn't scale well for APIs.

JWT is:

- Stateless
- Self-contained
- Secure
- Perfect for APIs

JWT Structure

xxxxx.yyyyy.zzzzz

Part	Purpose
Header	Algorithm + token type
Payload	User data (claims)
Signature	Integrity verification

JWT Authentication Flow

1. User logs in
2. Server validates credentials
3. Server generates JWT
4. Client stores JWT
5. Client sends JWT in every request
6. Server validates JWT

IMPLEMENTATION IN ASP.NET FRAMEWORK WEB API

Install Required Packages

Via NuGet:

```
System.IdentityModel.Tokens.Jwt  
Microsoft.IdentityModel.Tokens
```

JwtConfig.cs

```
public class JwtConfig  
{  
    public static string SecretKey = "THIS_IS_SUPER_SECRET_KEY_12345_A  
    BCDEFGHIJKLMNOPWXYZ";  
    public static string Issuer = "DemoIssuer";  
    public static string Audience = "DemoAudience";  
}
```

JwtTokenHelper.cs

```
public static class JwtTokenHelper
{
    public static string GenerateToken(UserModel user)
    {
        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(JwtConfig.SecretKey));
        var creds = new SigningCredentials(key, SecurityAlgorithms.Hmac
            Sha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.Name, user.Username),
            new Claim(ClaimTypes.Role, user.Role)
        };

        var token = new JwtSecurityToken(
            issuer: JwtConfig.Issuer,
            audience: JwtConfig.Audience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(60),
            signingCredentials: creds);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

JwtAuthorizationFilters.cs

```
public class JwtAuthorizeAttribute : AuthorizationFilterAttribute
{
    private string SecretKey = JwtConfig.SecretKey;

    public override void OnAuthorization(HttpContext actionContext)
    {
    }
```

```

var authHeader = actionContext.Request.Headers.Authorization;

if (authHeader == null || authHeader.Scheme != "Bearer")
{
    actionContext.Response =
        actionContext.Request.CreateResponse(HttpStatusCode.Un
authorized);
    return;
}

var token = authHeader.Parameter;
try
{
    var tokenHandler = new JwtSecurityTokenHandler();
    var key = Encoding.UTF8.GetBytes(SecretKey);

    var principal = tokenHandler.ValidateToken(token,
        new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(key),
            ClockSkew = TimeSpan.Zero
        },
        out _);

    actionContext.RequestContext.Principal = principal;
    Thread.CurrentPrincipal = principal;
}
catch
{
    actionContext.Response =
        actionContext.Request.CreateResponse(HttpStatusCode.Un
authorized);
}

```

```
}  
}
```

Now in AuthControlller on Login return the token
and then send that token as Bearer <token> in every request
Then use attributes like

```
[JwtAuthorize]  
for the controllers that needs to be authorized
```

For RBAC

use

```
[JwtAuthorize]  
[Authorize(Roles="Admin")]
```

▼ Filters

Filters are a CORE concept in ASP.NET Web API, and once you truly understand them, **auth, logging, validation, caching, exception handling** all become obvious.

REQUEST PIPELINE (BIG PICTURE)

```
HTTP Request  
  ↓  
Routing  
  ↓  
Authorization Filters  
  ↓  
Action Filters (Before)  
  ↓  
ActionMethod (Controller)  
  ↓  
Action Filters (After)  
  ↓  
Exception Filters (ifexception)  
  ↓  
HTTP Response
```


TYPES OF FILTERS

ASP.NET Web API has **4 main filter types**:

Filter Type	Interface / Base Class	When It Runs
Authorization Filter	<code>AuthorizationFilterAttribute</code>	Before everything
Action Filter	<code>ActionFilterAttribute</code>	Before & after action
Exception Filter	<code>ExceptionFilterAttribute</code>	When exception occurs
Result Filter	(MVC only)	Not in Web API

AUTHORIZATION FILTER

Purpose:

- Authentication
- Authorization
- Security checks

Base class:

```
AuthorizationFilterAttribute
```

Runs:

- **Before action**
- Even before action filters

JWT Filter Example

```
public class JwtAuthorizeAttribute : AuthorizationFilterAttribute
{
    public override void OnAuthorization(HttpContext actionContext)
    {
        // Validate token
        // Attach user
        // Reject if invalid
    }
}
```

```
}
```

Flow:

```
Request
↓
JwtAuthorizeAttribute
  → CheckAuthorizationheader
  → Validate JWT
  → Attach ClaimsPrincipal
↓
Controller
```

If you **set a response here**, the pipeline STOPS.

ACTION FILTER

Purpose:

- Logging
- Validation
- Timing
- Modifying request/response

Base class:

```
ActionFilterAttribute
```

Example: Logging Filter

```
public class LoggingFilter : ActionFilterAttribute
{
    public override void OnActionExecuting(HttpContext actionContext)
```

```

    {
    // Runs BEFORE controller
    }

    public override void OnActionExecuted(HttpActionExecutedContext context)
    {
    // Runs AFTER controller
    }
}

```

Execution:

Before → Action → After

Example: Validate Model

```

public class ValidateModelFilter : ActionFilterAttribute
{
    public override void OnActionExecuting(HttpActionContext actionContext)
    {
        if (!actionContext.ModelState.IsValid)
        {
            actionContext.Response =
                actionContext.Request.CreateErrorResponse(
                    HttpStatusCode.BadRequest,
                    actionContext.ModelState);
        }
    }
}

```

Stops execution if validation fails.

EXCEPTION FILTER

Purpose:

- Catch known exceptions
- Convert exceptions to HTTP responses

Base class:

ExceptionFilterAttribute

Example

```
public class ApiExceptionHandler : ExceptionFilterAttribute
{
    public override void OnException(HttpContext context)
    {
        context.Response =
            context.Request.CreateResponse(
                HttpStatusCode.BadRequest,
                new { message = "Something went wrong" });
    }
}
```

Runs **only** when exception occurs.

GLOBAL FILTERS (App_Start)

Applied to **ALL** controllers & actions.

```
config.Filters.Add(new JwtAuthorizeAttribute());
```

Best for:

- Logging
- Exception handling
- Global auth (rare)

CONTROLLER LEVEL

```
[JwtAuthorize]
public class ValuesController :ApiController
{
}
```

ACTION LEVEL

```
[JwtAuthorize]
public IActionResult Get()
{
}
```

▼ Exception Handling in APIs

Without proper handling:

- App crashes
- Clients get HTML error pages
- Security info leaks (stack traces)
- Inconsistent responses

A **good API**:

- Never crashes
- Always returns **JSON**
- Uses **proper HTTP status codes**
- Logs internally, hides details from client

Where Exceptions Happen

Examples:

- Null reference
- Invalid input

- Unauthorized access
- Database failure
- Business rule violation
- Unknown (500)

BAD WAY

```
public IHttpActionResult Get()
{
    try
    {
        // logic
    }
    catch (Exception ex)
    {
        return InternalServerError(ex);
    }
}
```

Repeating try/catch everywhere

Leaks internal details

Not scalable

Exception Handling Options in ASP.NET Web API

Technique	Use Case
<code>ExceptionHandlerAttribute</code>	Known / business exceptions
<code>IExceptionHandler</code>	Global fallback
<code>DelegatingHandler</code>	Low-level pipeline
Try/Catch in controller	Very rare

Define Standard Error Response

Create one consistent error format.

```
public class ApiError
{
    public int StatusCode {get;set; }
    public string Message {get;set; }
    public string ErrorCode {get;set; }
}
```

Custom Business Exception

```
public class BusinessException :Exception
{
    public string ErrorCode {get; }

    public BusinessException(string message,string errorCode ="BUSINESS
_ERROR")
        :base(message)
    {
        ErrorCode = errorCode;
    }
}
```

Used for:

- Rule violations
- Domain logic errors

Exception Filter (Used For Known Exceptions)

```
using System.Net;
using System.Net.Http;
using System.Web.Http.Filters;
```

```

public class ApiExceptionFilter : ExceptionFilterAttribute
{
    public override void OnException(HttpActionExecutedContext context)
    {
        if (context.Exception is BusinessException be)
        {
            context.Response = context.Request.CreateResponse(
                HttpStatusCode.BadRequest,
                new ApiError
                {
                    StatusCode = 400,
                    Message = be.Message,
                    ErrorCode = be.ErrorCode
                });
        }
    }
}

```

Global Exception Handler (Fallback)

```

using System.Net;
using System.Net.Http;
using System.Web.Http;
using System.Web.Http.ExceptionHandling;

public class GlobalExceptionHandler : ExceptionHandler
{
    public override void Handle(ExceptionHandlerContext context)
    {
        var response = context.Request.CreateResponse(
            HttpStatusCode.InternalServerError,
            new ApiError
            {
                StatusCode = 500,

```



```

        Message = "An unexpected error occurred",
        ErrorCode = "INTERNAL_SERVER_ERROR"
    });

    context.Result = new ResponseMessageResult(response);
}
}

```

Register Globally (App_Start)

```

public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        config.Filters.Add(new ApiExceptionHandler());

        config.Services.Replace(
            typeof(IExceptionHandler),
            new GlobalExceptionHandler());

        config.MapHttpAttributeRoutes();
    }
}

```

How Controllers Look Now(CLEAN)

```

[HttpGet]
public IHttpActionResult GetUser(int id)
{
    if (id <= 0)
        throw new BusinessException("Invalid user id","INVALID_ID");

    return Ok(new { Id = id });
}

```

No try/catch
Clean logic
Errors handled centrally

What Happens Under the Hood

```
Request
↓
Controller
↓
Exception thrown
↓
ExceptionHandler (known?)
↓
GlobalExceptionHandler (fallback)
↓
JSON response
```

▼ CACHING

Caching = storing a response so future requests can be served faster without recomputing.

Instead of:

```
Client → API → DB → API → Client
```

You get:

```
Client → Cache → Client
```

▼ HTTP CACHING

HTTP caching is a **client / proxy optimization** where responses are **stored** and **reused** instead of calling the server again.

The server **does NOT cache**

The server only **instructs** via headers

Core Header

Cache-Control

This tells:

- Browsers
- CDNs
- Proxies

Cache Locations

Depending on directives, data can be cached in:

- Browser memory
- Browser disk
- Reverse proxy (Nginx)
- CDN (Cloudflare, Akamai)
- Corporate / ISP proxy

Cache-Control Directives

Directive	Meaning
<code>public</code>	Anyone can cache
<code>private</code>	Only browser cache
<code>no-store</code>	Do NOT cache anywhere
<code>max-age</code>	Cache lifetime (seconds)

PUBLIC CACHE

Cache-Control:public

Example APIs

- Product catalog
- Country list
- Public config
- Landing page APIs

API Example (Public)

```
[HttpGet]
public IActionResult PublicProducts()
{
    var response = Request.CreateResponse(
        HttpStatusCode.OK,
        new { Time = DateTime.UtcNow });

    response.Headers.CacheControl = new CacheControlHeaderValue
    {
        Public = true,
        MaxAge = TimeSpan.FromMinutes(5)
    };

    return ResponseMessage(response);
}
```

PRIVATE CACHE

Cache-Control:private

Browser can cache

Proxy / CDN cannot cache

API Example

```
[Authorize]
[HttpGet]
```

```

public IHttpActionResult UserProfile()
{
    var response = Request.CreateResponse(
        HttpStatusCode.OK,
        new
        {
            User = User.Identity.Name,
            Time = DateTime.UtcNow
        });

    response.Headers.CacheControl = new CacheControlHeaderValue
    {
        Private = true,
        MaxAge = TimeSpan.FromSeconds(30)
    };

    return ResponseMessage(response);
}

```

NO-STORE

Meaning

Cache-Control:no-store

API Example (Login / Token

```

[HttpPost]
public IHttpActionResult Login(LoginModel model)
{
    var token = "JWT_TOKEN";

    var response = Request.CreateResponse(
        HttpStatusCode.OK,
        new { token });

    response.Headers.CacheControl = new CacheControlHeaderValue

```

```

    {
        NoStore =true,
        NoCache =true
    };

    return ResponseMessage(response);
}

```

▼ IN-MEMORY CACHE

| In-memory cache = data stored in RAM of the API process

- Lives inside **IIS worker process**
- Extremely fast (nanoseconds)
- Lost when app restarts

MemoryCache

Namespace:

```
using System.Runtime.Caching;
```

BASIC DEMO

```

var cache = MemoryCache.Default;

cache.Set(
    "greeting",
    "Hello World",
    DateTimeOffset.UtcNow.AddMinutes(1)
);

var value = cache.Get("greeting");

```

EXPIRATION TYPES

Absolute Expiration

Expires at fixed time.

```
var policy = new CacheItemPolicy
{
    AbsoluteExpiration = DateTimeOffset.UtcNow.AddMinutes(5)
};
```

Use when:

- Data changes at known interval

Sliding Expiration

Expires after inactivity.

```
var policy = new CacheItemPolicy
{
    SlidingExpiration = TimeSpan.FromMinutes(5)
};
```

Use when:

- Frequently accessed data

▼ API VERSIONING

Backward compatibility

1. Independent evolution
2. Clear contract
3. Minimal client changes

There are **5 industry-recognized techniques**.

URI PATH VERSIONING

```
/api/v1/users
/api/v2/users
```

QUERY STRING VERSIONING

```
/api/users?version=1
```

HEADER VERSIONING

```
X-API-Version:1
```

ACCEPT HEADER

```
Accept: application/vnd.myapp.v1+json
```

HOST / SUBDOMAIN VERSIONING

```
v1.api.myapp.com
```

▼ Swagger Documentation

Install Packages

```
Install-Package Swashbuckle
```

Enable XML Documentation

Project → Properties → Build

Check **XML documentation file**

Set path:

```
bin\SwaggerDemo.xml
```

Then document the model and classes using xml comments

Examples

```
/// <summary>  
/// Represents a product
```



```

/// </summary>
public class Product
{
    /// <summary>Product unique identifier</summary>
    public int Id { get; set; }
    /// <summary>Name of the product</summary>
    public string Name { get; set; }

    /// <summary>Product price</summary>
    public decimal Price { get; set; }

    /// <summary>Is product available</summary>
    public bool IsAvailable { get; set; }
}

```

```

using System.Collections.Generic;
using System.Web.Http;
using System.Web.Http.Description;
[RoutePrefix("api/products")]
public class ProductsController : ApiController
{
    /// <summary>
    /// Get all products
    /// </summary>
    /// <remarks>
    /// Returns the complete list of products from cache or database.
    /// </remarks>
    /// <returns>List of products</returns>
    [HttpGet]
    [Route("")]
    [ResponseType(typeof(List<Product>))]
    public IHttpActionResult GetAll()
    {
        var products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1200, IsAvailable = true },
            new Product { Id = 2, Name = "Phone", Price = 800, IsAvailable = true }
        }
    }
}

```

```

};

...

    return Ok(products);
}

/// <summary>
/// Get product by id
/// </summary>
/// <param name="id">Product identifier</param>
/// <returns>Product details</returns>
[HttpGet]
[Route("{id:int}")]
[ResponseType(typeof(Product))]
public IHttpActionResult GetById(int id)
{
    if (id <= 0)
        return BadRequest("Invalid product id");

    return Ok(new Product
    {
        Id = id,
        Name = "Laptop",
        Price = 1200,
        IsAvailable = true
    });
}

/// <summary>
/// Create a new product
/// </summary>
/// <param name="request">Product creation payload</param>
/// <returns>Created product</returns>
[HttpPost]
[Route("")]
[ResponseType(typeof(Product))]
public IHttpActionResult Create(ProductCreateRequest request)
{

```

```

        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var product = new Product
        {
            Id = 3,
            Name = request.Name,
            Price = request.Price,
            IsAvailable = true
        };

        return Created("api/products/3", product);
    }

    /// <summary>
    /// Delete product
    /// </summary>
    /// <param name="id">Product identifier</param>
    /// <returns>No content</returns>
    [HttpDelete]
    [Route("{id:int}")]
    [ResponseType(typeof(void))]
    public IHttpActionResult Delete(int id)
    {
        return Ok();
    }

    ...

}

```

Then Add this to the swagger config

```

using System;
using System.Web.Http;
using Swashbuckle.Application;
using WebActivatorEx;
[assembly: PreApplicationStartMethod(typeof(SwaggerConfig), "Regist

```

```

er"")]

public class SwaggerConfig
{
    public static void Register()
    {
        GlobalConfiguration.Configuration
            .EnableSwagger(c =>
            {
                c.SingleApiVersion("v1", "Products API");

                ...

                c.IncludeXmlComments(
                    AppDomain.CurrentDomain.BaseDirectory + @"\\bin\\Swagger
                    Demo.xml"
                );
            })
            .EnableSwaggerUi();
    }

    ...

}

```

▼ Dependency Injection

WHAT IS DEPENDENCY?

A **dependency** is any object that another object needs to do its work.

Example

```

class OrderService
{
    private EmailService _email; // dependency
}

```

`OrderService` depends on `EmailService`

WHAT IS DEPENDENCY INJECTION (DI)?

Dependency Injection is a design pattern where:

Dependencies are provided from the outside instead of being created inside the class.

Without DI (Tightly Coupled)

```
class OrderService
{
    private EmailService _email =new EmailService();
}
```

Problems:

- Hard to test
- Hard to replace
- Tight coupling

With DI (Loosely Coupled)

```
class OrderService
{
    private IEmailService _email;

    public OrderService(IEmailService email)
    {
        _email = email;
    }
}
```

Benefits:

- Easy testing
- Easy replacement

- Loose coupling

TYPES OF DEPENDENCY INJECTION (VERY IMPORTANT)

Constructor Injection

```
class OrderController
{
    private IOrderService _service;

    public OrderController(IOrderService service)
    {
        _service = service;
    }
}
```

Mandatory dependency

Compile-time safety

Most preferred

Property Injection

```
class OrderController
{
    public IOrderService Service {get;set; }
}
```

Optional dependency

Can be forgotten

Rarely used

Method Injection

```

class OrderService
{
    public void Process(IEmailService email)
    {
        email.Send();
    }
}

```

Used when dependency is required only for one method

WHAT IS A DI CONTAINER?

A **DI Container** is responsible for:

- Creating objects
- Injecting dependencies
- Managing lifetimes

Popular Containers

Framework	Container
ASP.NET Core	Built-in
ASP.NET Framework	Unity, Autofac, Ninject

SERVICE & REPOSITORY – COMPLETE FLOW

Controller → Service → Repository → DB

```

Controller
  ↓
Service (Business Logic)
  ↓
Repository (Data Access)
  ↓
Database

```

Repository

```
public interface IOrderRepository
{
    string GetOrders();
}

public class OrderRepository : IOrderRepository
{
    public string GetOrders() ⇒ "Orders from DB";
}
```

Service

```
public interface IOrderService
{
    string GetOrders();
}

public class OrderService : IOrderService
{
    private readonly IOrderRepository _repo;

    public OrderService(IOrderRepository repo)
    {
        _repo = repo;
    }

    public string GetOrders() ⇒ _repo.GetOrders();
}
```

Controller

```
public class OrdersController : ApiController
{
    private readonly IOrderService _service;
```



```
public OrdersController(IOrderService service)
{
    _service = service;
}

public IHttpActionResult Get()
{
    return Ok(_service.GetOrders());
}
}
```

LIFETIMES IN DEPENDENCY INJECTION

TRANSIENT

Definition

| New instance every time it is requested

Example

```
container.RegisterType<IService, Service>();
```

Behavior

- New instance per resolve
- Lightweight objects
- Stateless services

Controllers

Validators

DbContext

SCOPED (REQUEST-BASED)

Definition

| One instance per request

IMPORTANT (FRAMEWORK vs CORE)

Platform	Scoped
ASP.NET Core	Real
ASP.NET Framework	Same as Transient unless configured

Unity (Framework)

```
container.RegisterType<IService, Service>(
    new HierarchicalLifetimeManager());
```

DbContext

Repository

Service

SINGLETON

Definition

| One instance for entire application

```
container.RegisterSingleton<IService, Service>();
```

Use Cases

Caching

Logging

Config

DbContext

HttpContext

LIFETIME COMPARISON TABLE

Lifetime	Instance Created	Best Use
Transient	Every resolve	Controllers
Scoped	Per request	DbContext
Singleton	App lifetime	Cache

▼ Logging

Logging = recording application events so that you can:

- Debug issues
- Track errors
- Monitor behavior
- Audit user actions
- Analyze performance

Without logging:

- Production bugs are **impossible** to debug
- You only see:

An error has occurred

With logging:

- You see **what, where, why**
- You can fix issues without reproducing locally

TYPES OF LOGGING

A. By SEVERITY (LOG LEVELS)

Level	Use
Trace	Very detailed (dev only)
Debug	Debugging info
Information	Normal flow
Warning	Something odd
Error	Exception occurred

Level	Use
Fatal	App crash

B. By LOCATION

Where	Example
Controller	Request data
Service	Business logic
Repository	DB issues
Global	Unhandled exceptions
Middleware	Request/response

C. By TARGET (WHERE LOGS GO)

Target	Example
File	<code>.txt</code> , <code>.log</code>
Database	SQL table
Console	Debug
Event Viewer	Windows
Cloud	Azure App Insights

4. LOGGING IN ASP.NET WEB API – OPTIONS

Library
log4net
NLog
Serilog

To Setup NLog

INSTALL EXACT PACKAGES

Open **Package Manager Console**

```
Install-Package NLog-Version5.2.0
Install-Package NLog.Web-Version5.2.0
Install-Package NLog.Config-Version5.2.0
```

ADD NLog.config

Add file at PROJECT ROOT

NLog.config

```
<?xml version="1.0" encoding="utf-8" ?>
<nlogxmlns="http://www.nlog-project.org/schemas/NLog.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
autoReload="true"
throwConfigExceptions="true"
internalLogLevel="Trace"
internalLogFile="C:\Logs\nlog-internal.log">

  <targets>
    <targetname="file"
xsi:type="File"
fileName="C:\Logs\api-${shortdate}.log"
layout="${longdate}|${level:uppercase=true}|${logger}|${message}|
${exception:format=ToString}" />
  </targets>

  <rules>
    <loggername="*"minlevel="Trace"writeTo="file" />
  </rules>

</nlog>
```

CRITICAL CHECKS

File name is **exactly** `NLog.config`

Build Action = `Content`

Copy to Output Directory = Copy if newer

Folder C:\Logs exists

App has permission to write

FORCE NLOG TO Global.asax.cs

```
using NLog;
using System.Web.Http;

namespace NLogFullDemo
{
    public class WebApiApplication : System.Web.HttpApplication
    {
        private static readonly Logger Logger =
            LogManager.GetCurrentClassLogger();

        protected void Application_Start()
        {
            LogManager.LoadConfiguration("NLog.config");

            GlobalConfiguration.Configure(WebApiConfig.Register);

            Logger.Info("Application started");
        }

        protected void Application_End()
        {
            Logger.Info("Application stopped");
            LogManager.Shutdown();
        }
    }
}
```

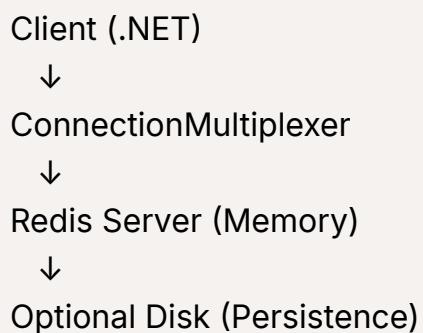
▼ Redis

Redis = Remote Dictionary Server

- In-memory data store
- Key-value based
- Extremely fast (microseconds)
- Can persist data to disk
- Used as:
 - Cache
 - Database
 - Message broker

Use Case	Example
Caching	Products, Users
Session store	Login sessions
Rate limiting	API throttling
Queue	Background jobs
Leaderboard	Game scores
Pub/Sub	Notifications
Distributed lock	Prevent duplicates

REDIS ARCHITECTURE



REDIS CONNECTION IN .NET

ConnectionMultiplexer

```
ConnectionMultiplexer.Connect("localhost:6379");
```

- Thread-safe
- Heavy object
- **One per application**

Correct Pattern

```
static Lazy<ConnectionMultiplexer>
```

REDIS DATABASES

- Logical DBs: **0-15**
- Same server, different namespaces

```
GetDatabase(0);  
GetDatabase(1);
```

REDIS KEY RULES

- Keys are strings
- Use naming convention:

```
product:1  
product:all  
user:5  
session:token
```

REDIS DATA TYPES

STRING

Description

- Simple key → value
- Max size: 512MB

Commands

```
StringSet  
StringGet  
StringIncrement
```

Example

```
db.StringSet("name","John");  
db.StringGet("name");
```

Use Cases

- Cache
- Counters
- Flags

HASH (OBJECT STORAGE)

Description

- Key → multiple fields
- Best for objects

Example

```
db.HashSet("user:1",new[]  
{  
    new HashEntry("Name","Alex"),  
    new HashEntry("Age",25)  
});
```

Use Cases

- Users
- Products
- Sessions

LIST (QUEUE / STACK)

Description

- Ordered collection
- Allows duplicates

Commands

```
ListLeftPush  
ListRightPop
```

Use Cases

- Job queue
- FIFO / LIFO

SET (UNIQUE COLLECTION)

Description

- Unordered
- Unique values only

Commands

```
SetAdd  
SetContains
```

Use Cases

- Online users
- Permissions

- Tags

SORTED SET (RANKING)

Description

- Like set but with score

Example

```
SortedSetAdd("scores","user1",100);
```

Use Cases

- Leaderboards
- Rankings
- Priority queues

KEY EXPIRATION (TTL)

Description

- Auto delete keys

Example

```
StringSet("key","value", TimeSpan.FromMinutes(10));
```

Use Cases

- Cache expiry
- Sessions